



BITS Pilani
Pilani Campus

Digital Design

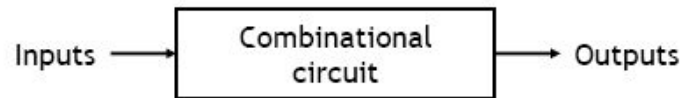
Prof. GSS Chalapathi,
Prof. Saurabh Gandhi,
Prof. Ramesh Sambangi

Review questions

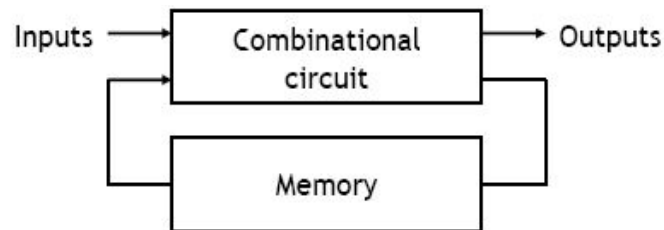


1. Arrays in Verilog
2. Data types in verilog

Sequential circuits



- So far we've only worked with **combinational circuits**, where applying the same inputs always produces the same outputs.
 - This corresponds to a mathematical function, where every input has a single, unique output.
 - In programming terminology, combinational circuits are similar to "functional programs" that do not contain variables and assignments.
- Such circuits are comparatively easy to design and analyze.



- In contrast, the outputs of a **sequential circuit** depend on not only the inputs, but also the **state**, or the current contents of some memory.
- This makes things more difficult to understand since the same inputs can yield *different* outputs, depending on what's stored in memory.
- The memory contents can also change as the circuit runs, so the *order* in which things occur makes a difference.

Examples of sequential devices

- Many real-life devices are sequential in nature.
 - Combination locks open if you enter numbers in the right order.
 - Elevators move up or down and open or close in response to buttons that are pressed on different floors and in the elevator itself.
 - Traffic lights change from red to green depending on whether a car is waiting at the intersection.
- More importantly for us, computers are sequential! For instance, key presses and mouse clicks have different effects based on which program is loaded into memory, and the state of that program.



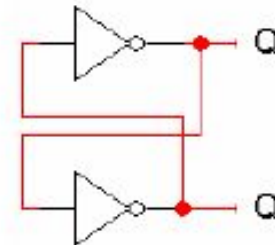
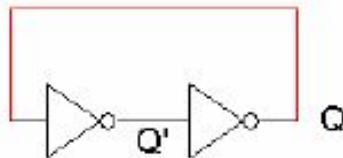
What is memory?

- A memory should support at least three operations.
 1. It should be able to hold a value.
 2. You should be able to *read* the value that is saved.
 3. You should be able to *change* that value.
- We'll start with the simplest case, a one-bit memory.
 1. It should be able to hold a single bit, 0 or 1.
 2. You should be able to read the bit that is saved.
 3. You should be able to change the bit.
 - You can **set** the bit to 1
 - You can **reset** or **clear** the bit to 0.

Storing a bit in a digital device



- How can a circuit remember anything, when it's just a bunch of gates that produce outputs according to the inputs?
- The idea is to make a loop in a circuit, so the outputs are *also* inputs.
- Here is one initial attempt, shown with two equivalent layouts.

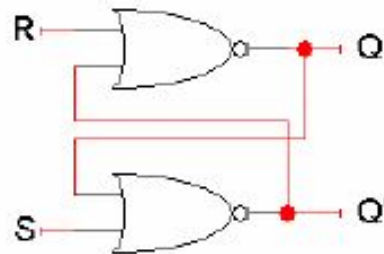


- Does this satisfy the properties of memory?
 - These circuits “remember” Q since its value never changes. Similarly, Q' never changes either.
 - We can “read” Q by sending it to another gate or device, as usual.
 - But we can’t *change* Q ! There are no external inputs here, so we can’t control whether $Q=1$ or $Q=0$.

Storing a *changeable* bit in a device



- Let's use NOR gates instead of inverters. The **SR latch** here has two inputs S and R, which will let us control the outputs Q and Q'.



- Q and Q' feed back into the circuit, so they're not only outputs, they're also inputs!
- To figure out how Q and Q' change, we must look at not only the inputs S and R, but also the *current* values of Q and Q'.

$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$

$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$

- Let's see how different input values for S and R affect this thing.

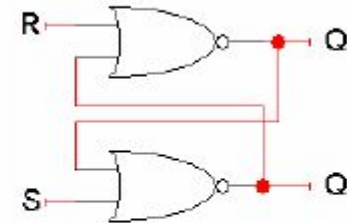
Storing a value (SR = 00)



- What if $S = 0$ and $R = 0$?
- The equations on the right reduce to:

$$Q_{\text{next}} = (0 + Q'_{\text{current}})' = Q_{\text{current}}$$
$$Q'_{\text{next}} = (0 + Q_{\text{current}})' = Q'_{\text{current}}$$

- So when $SR = 00$, then $Q_{\text{next}} = Q_{\text{current}}$.
- This is exactly what we need to **store** values in the latch.



$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$
$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$

Setting the latch (SR = 10)

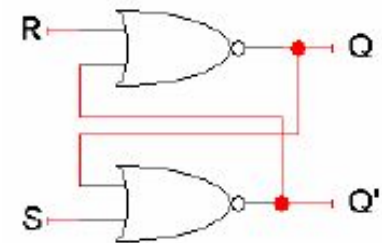
- What if $S = 1$ and $R = 0$?
- Since $S = 1$, Q'_{next} is 0, *regardless* of Q_{current} .

$$Q'_{\text{next}} = (1 + Q_{\text{current}})' = 0$$

- Then this new value of Q' goes into the top NOR gate, along with $R = 0$.

$$Q_{\text{next}} = (0 + 0)' = 1$$

- So when $SR = 10$, then $Q'_{\text{next}} = 0$ and $Q_{\text{next}} = 1$. This is how you **set** the latch to 1; the S input stands for "set."
- Notice it can take up to two steps (two gate delays) from the time S becomes 1 to the time Q_{next} becomes 1.
- But once Q_{next} becomes 1, the outputs will stop changing. This is a **stable state**.



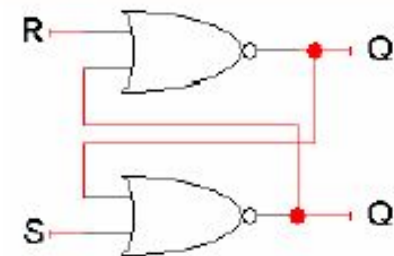
$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$

$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$

Latch delays



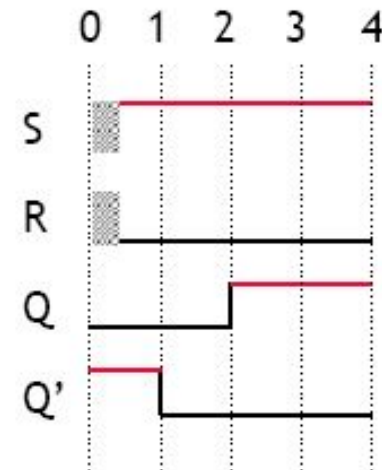
- Timing diagrams are especially useful for seeing how sequential circuits work.
- Here is a diagram which shows an example of how our latch outputs change with inputs $SR = 10$.



0. Let's say that $Q = 0$ and $Q' = 1$ initially.

1. Since $S = 1$, Q' changes from 1 to 0 after one NOR-gate delay (marked with vertical lines in the timing diagram).
2. This change in Q' , along with $R = 0$, causes Q to become 1 after another gate delay.
3. The latch then stabilizes until S or R change again.

$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$
$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$



Resetting the latch (SR = 01)

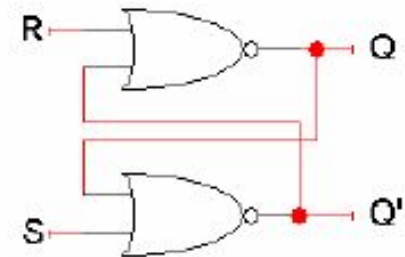
- What if $S = 0$ and $R = 1$?
- Since $R = 1$, Q_{next} is 0, *regardless of* Q_{current} .

$$Q_{\text{next}} = (1 + Q'_{\text{current}})' = 0$$

- Then this new value of Q goes into the bottom NOR gate, where $S = 0$.

$$Q'_{\text{next}} = (0 + 0)' = 1$$

- So when $SR = 01$, then $Q_{\text{next}} = 0$ and $Q'_{\text{next}} = 1$. This is how you **reset**, or **clear**, the latch to 0; the R input stands for "reset."
- Again, it can take two gate delays before a change in R propagates to the output Q'_{next} .



$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$

$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$

SR latch forms a memory unit

- This **characteristic table** shows that our latch provides everything we need in a memory: we can set it, reset it, or keep the current value.
- The output Q represents the data stored in the latch. It is also called the **state** of the latch.
- We can expand the table above into a **state table**, which explicitly shows that the *next* values of Q and Q' depend on their *current* values, as well as on the inputs S and R.

S	R	Q
0	0	No change
0	1	0 (reset)
1	0	1 (set)

Inputs		Current		Next	
S	R	Q	Q'	Q	Q'
0	0	0	1	0	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0

SR latches are sequential

- Note that for $SR = 00$, the next value of Q could be *either* 0 or 1, depending on the current value of Q .
- So the same inputs can produce different outputs, depending on whether the latch is currently set or reset.
- This is different from the combinational circuits that we've seen so far, where the same inputs always generate the same outputs.

S	R	Q
0	0	No change
0	1	0 (reset)
1	0	1 (set)

Inputs		Current		Next	
S	R	Q	Q'	Q	Q'
0	0	0	1	0	1
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	0	0	1
1	0	0	1	1	0
1	0	1	0	1	0

What happens when SR = 11?

- Both Q_{next} and Q'_{next} would become 0, which contradicts the assumption that Q and Q' are always complements.
- ~~Another~~ ^{The} problem is what happens if we then make $S = 0$ and $R = 0$ together.

$$Q_{\text{next}} = (0 + 0)' = 1$$

$$Q'_{\text{next}} = (0 + 0)' = 1$$

- But these new values go back into the NOR gates, and we then get $Q = Q' = 0$ again.

$$Q_{\text{next}} = (0 + 1)' = 0$$

$$Q'_{\text{next}} = (0 + 1)' = 0$$

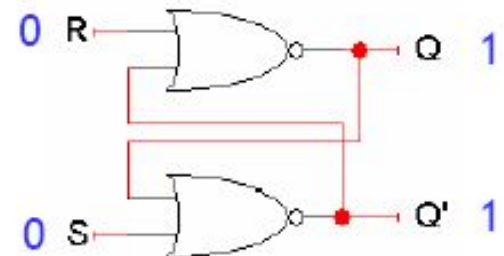
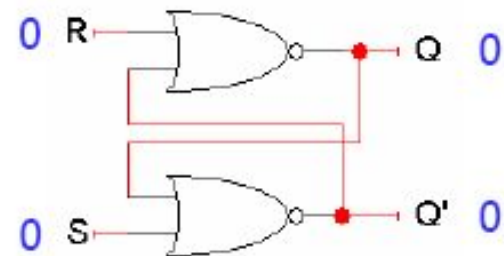
- So the circuit enters an infinite loop, where Q and Q' cycle between 0 and 1 forever.
- ~~Don't set SR = 11!~~

Really?

This is the real purpose of your lab, go try it out!

$$Q_{\text{next}} = (R + Q'_{\text{current}})'$$

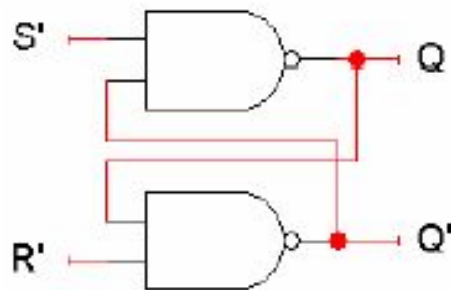
$$Q'_{\text{next}} = (S + Q_{\text{current}})'$$



S'R' latch



- There are several other variations of the basic latch.
- You can use NAND instead of NOR gates to get a **S'R' latch**.

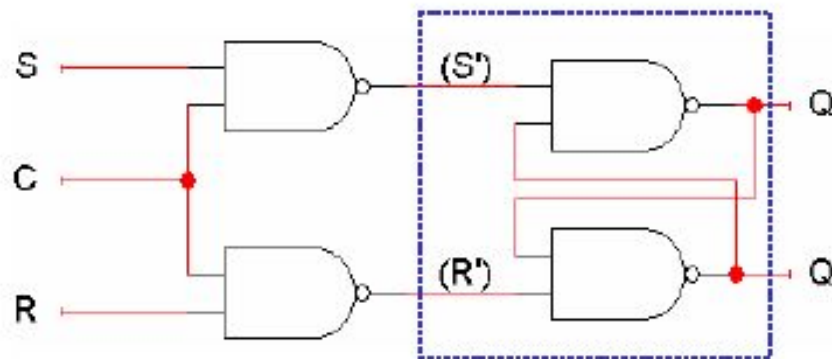


S'	R'	Q
1	1	No change
1	0	0 (reset)
0	1	1 (set)
0	0	Avoid!

- This is just like an SR latch but with inverted inputs, as you can see from the table.

SR latch with control input

- Here is an SR latch with a **control input** C , which acts like an enable.



C	S	R	S'	R'	Q
0	x	x	1	1	No change
1	0	0	1	1	No change
1	0	1	1	0	0 (reset)
1	1	0	0	1	1 (set)
1	1	1	0	0	Evil!

- Notice the hierarchical design!
 - The dotted blue box contains the $S'R'$ latch from the previous slide.
 - The additional NAND gates are simply used to generate appropriate inputs for the $S'R'$ latch.